

Laporan Praktikum Kontrol Cerdas

IP-CAM & MQTT

Nama : Dido Akant Pranuris

NIM : 224308067

Kelas : TKA-6C

Akun Github (Tautan) : <https://github.com/pranurisddo-jpg>

Abstrak

Perkembangan teknologi Internet of Things (IoT) memungkinkan berbagai perangkat untuk saling terhubung dan bertukar data secara real-time melalui jaringan internet. Pada praktikum ini dilakukan implementasi sistem monitoring berbasis computer vision dengan memanfaatkan IP Camera, OpenCV, MediaPipe, dan protokol komunikasi MQTT. IP Camera digunakan sebagai sumber video streaming yang diakses melalui jaringan menggunakan alamat IP, sedangkan OpenCV digunakan untuk menampilkan dan memproses citra video secara real-time. Proses deteksi objek dilakukan menggunakan MediaPipe dengan metode deteksi tangan yang mampu mengenali keberadaan tangan berdasarkan landmark yang terdeteksi pada citra. Hasil deteksi kemudian dikirimkan melalui protokol MQTT menggunakan konsep publish dan subscribe dengan perantara broker sehingga dapat diterima oleh perangkat lain. Berdasarkan hasil percobaan, sistem mampu melakukan deteksi tangan secara real-time serta mengirimkan informasi hasil deteksi berupa status keberadaan tangan melalui jaringan internet. Integrasi antara teknologi computer vision dan protokol MQTT menunjukkan bahwa sistem ini dapat digunakan sebagai dasar pengembangan aplikasi monitoring berbasis IoT yang lebih kompleks.

Kata kunci: IP Camera, OpenCV, MediaPipe, MQTT, Intern

1. Pendahuluan :

IP Camera (Internet Protocol Camera) merupakan kamera digital yang mampu mengirimkan dan menerima data melalui jaringan berbasis Internet Protocol (IP). Berbeda dengan sistem kamera analog konvensional yang memerlukan perangkat tambahan seperti Digital Video Recorder (DVR), IP Cam dapat mentransmisikan data video secara langsung melalui jaringan lokal maupun internet. Hal ini menjadikan IP Cam lebih fleksibel untuk sistem pengawasan jarak jauh dan aplikasi berbasis pengolahan citra digital (Kurose and

Ross, 2017). Dalam implementasi computer vision, IP Cam sering digunakan sebagai sumber input visual untuk proses analisis citra menggunakan perangkat lunak seperti OpenCV.

Dalam arsitektur IoT, komunikasi data antar perangkat memerlukan protokol yang ringan, efisien, dan mampu bekerja pada jaringan dengan bandwidth terbatas. Message Queuing Telemetry Transport (MQTT) merupakan protokol komunikasi berbasis publish/subscribe yang dirancang untuk perangkat dengan sumber daya terbatas dan konektivitas jaringan yang tidak stabil. MQTT bekerja menggunakan perantara yang disebut broker, yang bertugas menerima pesan dari publisher dan mendistribusikannya kepada subscriber berdasarkan topik tertentu (Banks and Gupta, 2014).

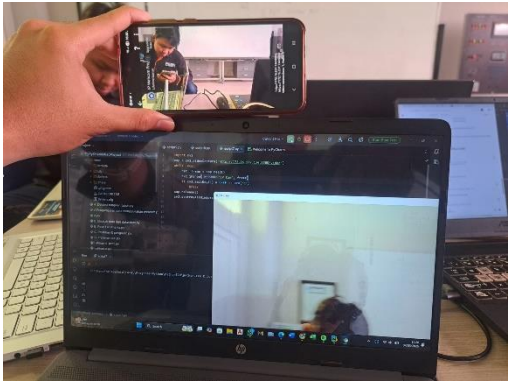
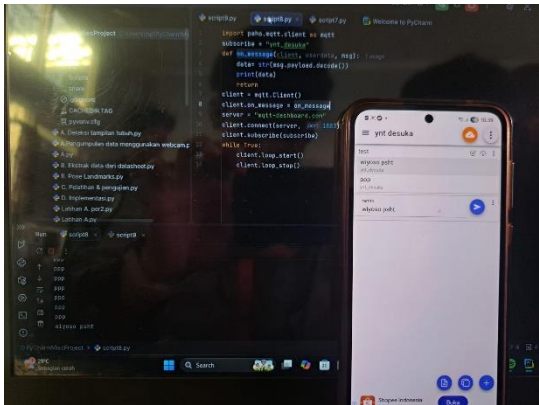
2. Tujuan Percobaan :

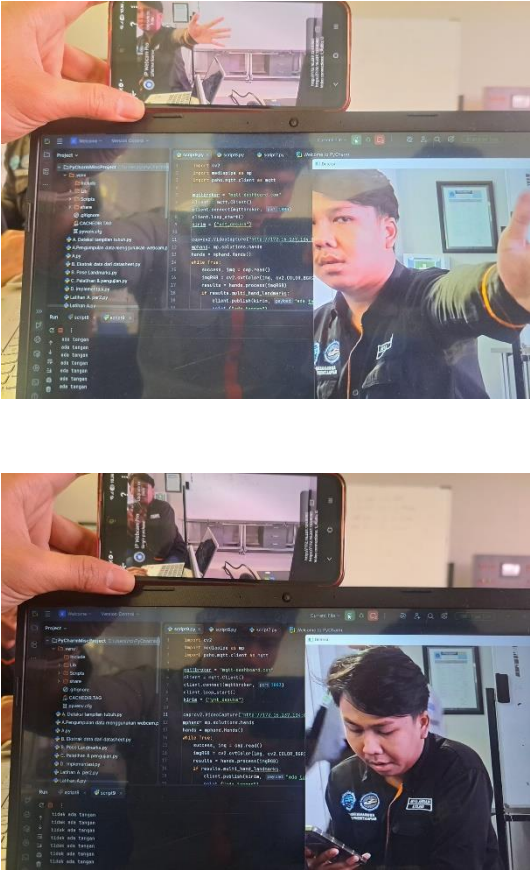
- Memahami konsep kerja IP Camera sebagai sumber data visual berbasis jaringan dalam sistem monitoring real-time.
- Mempelajari penggunaan library OpenCV untuk menampilkan dan mengolah citra digital dari IP Cam.
- Mengimplementasikan metode deteksi tangan menggunakan MediaPipe untuk mengenali keberadaan objek tangan secara real-time.
- Memahami mekanisme komunikasi data menggunakan protokol MQTT berbasis publish/subscribe.

3. Manfaat Percobaan :

- Memberikan pemahaman praktis mengenai integrasi antara computer vision dan Internet of Things (IoT).
- Melatih kemampuan dalam pengolahan citra digital menggunakan OpenCV dan MediaPipe.
- Meningkatkan pemahaman tentang protokol MQTT sebagai media komunikasi ringan untuk sistem IoT.
- Mengembangkan keterampilan dalam membangun sistem monitoring real-time berbasis jaringan.

4. Data dan Output Hasil Percobaan :

No	Variabel	Hasil Pengamatan
1.	A. Akses IP-CAM	 <pre>1 import cv2 2 cap = cv2.VideoCapture("http://172.16.237.134:8080/video") 3 while True: 4 ret, frame = cap.read() 5 cv2.imshow('IP Cam', frame) 6 if cv2.waitKey(1) & 0xFF == ord('q'): 7 break 8 cap.release() 9 cv2.destroyAllWindows()</pre>
2.	B. MQTT	

		<pre>1 import paho.mqtt.client as mqtt 2 subscribe = "ynt_desuka" 3 def on_message(client, userdata, msg): 4 data= str(msg.payload.decode()) 5 print(data) 6 return 7 client = mqtt.Client() 8 client.on_message = on_message 9 server = "mqtt-dashboard.com" 10 client.connect(server, 1883) 11 client.subscribe(subscribe) 12 while True: 13 client.loop_start() 14 client.loop_stop()</pre>
3.	C. Publish MQTT (ada/tidak ada tangan)	

		<pre> 1 import cv2 2 import mediapipe as mp 3 import paho.mqtt.client as mqtt 4 5 mqttbroker = "mqtt-dashboard.com" 6 client = mqtt.Client() 7 client.connect(mqttbroker, 1883) 8 client.loop_start() 9 kirim = ("ynt_desuka") 10 11 cap=cv2.VideoCapture("http://172.16.237.134:8080/video") 12 mphand= mp.solutions.hands 13 hands = mphand.Hands() 14 while True: 15 success, img = cap.read() 16 imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) 17 results = hands.process(imgRGB) 18 if results.multi_hand_landmarks: 19 client.publish(kirim, "ada tangan") 20 print ("ada tangan") 21 else: 22 client.publish(kirim, "tidak ada tangan") 23 print ("tidak ada tangan") 24 cv2.imshow("Deteksi", img) 25 if cv2.waitKey(1) & 0xFF == ord('q'): 26 break 27 28 </pre>
--	--	---

5. Analisis Program :

- Akses IP-CAM: Program pertama digunakan untuk menampilkan video streaming dari IP Camera menggunakan library OpenCV. Pada program ini kamera diakses melalui alamat IP berupa URL stream yang dihasilkan oleh aplikasi IP Webcam pada smartphone. Perintah `cv2.VideoCapture()` digunakan untuk membuka sumber video dari alamat IP yang telah ditentukan. Selanjutnya program membaca frame video secara terus-menerus menggunakan fungsi `cap.read()`. Frame yang berhasil diambil kemudian ditampilkan pada jendela menggunakan fungsi `cv2.imshow()`. Proses pembacaan frame dilakukan secara berulang dalam sebuah *perulangan while True*, sehingga video dapat ditampilkan secara real-time. Fungsi `cv2.waitKey(1)` digunakan untuk memberikan jeda kecil pada proses pembacaan frame sekaligus membaca input dari keyboard. Program akan berhenti ketika tombol q ditekan. Setelah program selesai dijalankan, kamera dilepaskan menggunakan `cap.release()` dan semua jendela OpenCV ditutup dengan `cv2.destroyAllWindows()`.
- MQTT : Program kedua berfungsi sebagai penerima data menggunakan protokol MQTT. Library `paho.mqtt.client` digunakan untuk menghubungkan program

Python dengan broker MQTT. Pada program ini ditentukan sebuah topik (topic) yang akan digunakan sebagai jalur komunikasi data, yaitu *ynt_desuka*. Fungsi *on_message()* dibuat untuk menangani pesan yang diterima dari broker MQTT. Ketika sebuah pesan diterima, payload pesan akan diubah menjadi bentuk string menggunakan *msg.payload.decode()* dan kemudian ditampilkan pada terminal menggunakan fungsi *print()*. Program kemudian membuat objek client MQTT menggunakan *mqtt.Client()* dan menghubungkannya ke server broker melalui perintah *client.connect()*. Setelah berhasil terhubung, client akan melakukan subscribe terhadap topik yang telah ditentukan sehingga dapat menerima setiap pesan yang dikirim oleh publisher pada topik tersebut.

- c. Publish MQTT : Program ketiga merupakan program utama yang mengintegrasikan pengolahan citra dengan komunikasi IoT. Program ini menggunakan tiga library utama yaitu OpenCV untuk pengolahan citra, MediaPipe untuk deteksi tangan, serta Paho MQTT untuk komunikasi data. Pertama, program membuat koneksi ke broker MQTT menggunakan *mqtt.Client()* dan *client.connect()*. Topik yang digunakan untuk mengirim data ditentukan pada variabel *kirim*. Setelah itu program membuka kamera menggunakan *cv2.VideoCapture()* untuk mengambil frame video secara real-time. Frame yang diambil dari kamera kemudian dikonversi dari format warna BGR ke RGB menggunakan *cv2.cvtColor()*. Konversi ini diperlukan karena MediaPipe memproses citra dalam format RGB. Selanjutnya frame tersebut diproses oleh modul *hands.process()* untuk mendeteksi apakah terdapat tangan pada gambar. Jika MediaPipe mendeteksi landmark tangan (*results.multi_hand_landmarks*), maka program akan mengirimkan pesan "ada tangan" ke broker MQTT menggunakan fungsi *client.publish()*. Sebaliknya, jika tidak terdeteksi tangan maka program akan mengirimkan pesan "tidak ada tangan". Selain itu status deteksi juga ditampilkan pada terminal menggunakan perintah *print()*.

6. Kesimpulan :

- IP Camera dapat digunakan sebagai sumber video streaming yang dapat diakses melalui jaringan menggunakan alamat IP, sehingga memungkinkan pengambilan citra secara real-time pada komputer.
- Library OpenCV dapat digunakan untuk menampilkan dan memproses video streaming dari IP Camera sehingga frame video dapat diolah lebih lanjut untuk keperluan analisis citra.
- MediaPipe mampu melakukan deteksi tangan secara real-time dengan memanfaatkan landmark pada bagian tangan yang terdeteksi pada frame video.
- Protokol MQTT dapat digunakan sebagai media komunikasi data yang ringan dan efisien dalam sistem Internet of Things (IoT) dengan menggunakan konsep publish dan subscribe melalui broker.
- Integrasi antara OpenCV, MediaPipe, dan MQTT memungkinkan sistem untuk mendeteksi keberadaan tangan pada video dan mengirimkan hasil deteksi tersebut ke perangkat lain secara real-time melalui jaringan internet.

7. Saran

Berdasarkan hasil percobaan yang telah dilakukan, terdapat beberapa hal yang dapat diperhatikan untuk pengembangan dan peningkatan sistem di masa mendatang. Pertama, kualitas koneksi jaringan perlu diperhatikan karena penggunaan IP Camera dan komunikasi MQTT sangat bergantung pada kestabilan jaringan. Koneksi internet yang tidak stabil dapat menyebabkan keterlambatan dalam pengiriman data maupun gangguan pada proses streaming video. Kedua, resolusi video dari IP Camera sebaiknya disesuaikan dengan kemampuan perangkat yang digunakan. Penggunaan resolusi yang terlalu tinggi dapat meningkatkan beban pemrosesan pada komputer sehingga mempengaruhi performa sistem dalam melakukan deteksi objek secara real-time. Oleh karena itu, pengaturan resolusi yang lebih rendah atau proses resize frame dapat menjadi solusi untuk meningkatkan efisiensi pemrosesan.

8. Referensi

Belajar Python– Situs Open Source Tutorial Pemrograman Python Bahasa Indonesia

<https://belajarpython.com>

Kurose, J.F. and Ross, K.W., 2017. *Computer networking: a top-down approach*. 7th ed.

Boston: Pearson Education.